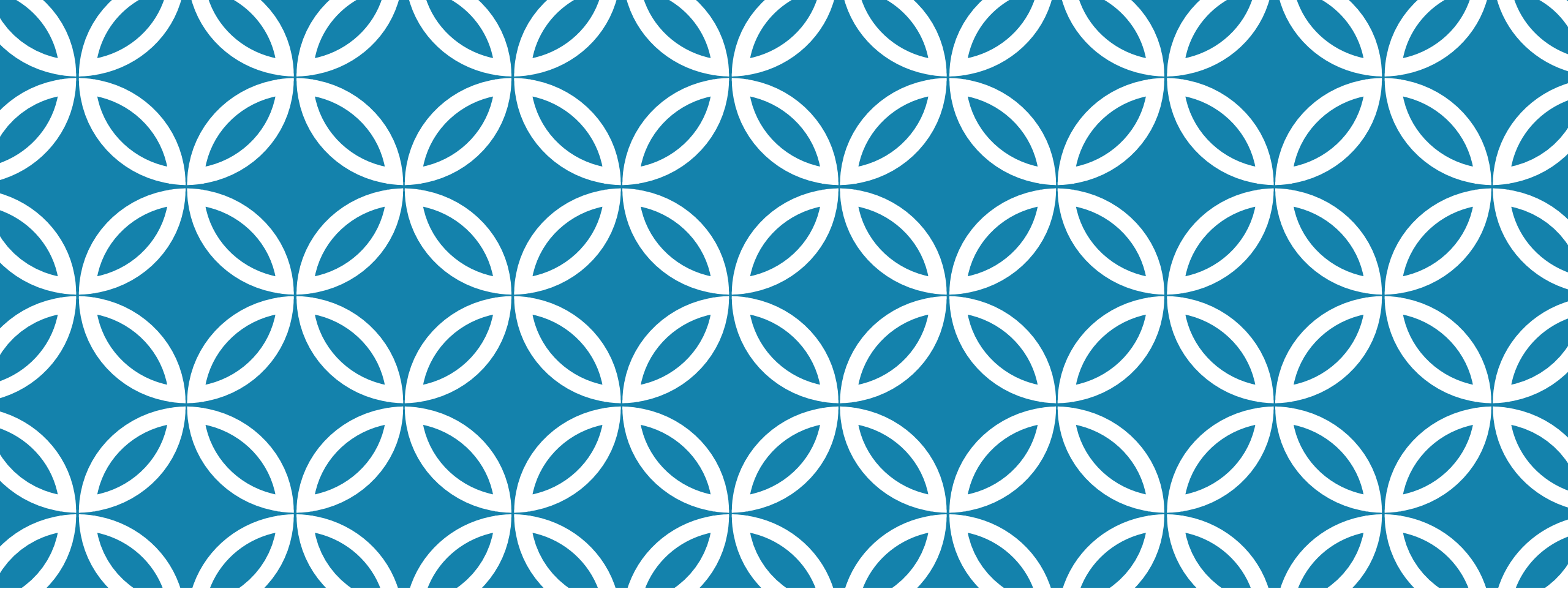




DATA SCIENCE

sec6

TENSORFLOW

Building a machine learning model in TensorFlow typically involves the following steps:

Step 1: Define the Model

- Create a model by specifying the layers, activation functions, and output layers.
- Choose an optimizer, loss function, and evaluation metric.

Step 2: Prepare the Data

- Load, preprocess, and split the dataset into training, validation, and test sets.
- Data preprocessing might include normalization, reshaping, and encoding categorical variables.

TENSORFLOW

Step 3: Train the Model

- Feed the training data into the model and perform forward and backward passes.
- Use optimization algorithms such as stochastic gradient descent (SGD) or Adam to minimize the loss function.

Step 4: Evaluate the Model

- After training, evaluate the model's performance on the validation and test sets to assess its generalization ability.

Step 5: Deploy the Model

- Once the model is trained and evaluated, deploy it to a production environment using TensorFlow Serving, TensorFlow Lite, or other deployment tools.

TENSORFLOW

- Building a Simple Model with TensorFlow
- Let's learn how to create and train a simple neural network with TensorFlow using the steps discussed above.
- Here, we have loaded the MNIST Dataset and processed the image. Then, we have built a simple neural network using TensorFlow's Sequential API with two layers:
 - dense layer with ReLU activation
 - an output layer with softmax activation function
- At last we compiled the model using Adam Optimizer and Sparse Categorical Crossentropy, and trained the model for 5 epochs.

TENSORFLOW

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.datasets import mnist

# Load the MNIST dataset
(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

# Preprocess the data: flatten the images and normalize the pixel values
train_images = train_images.reshape((train_images.shape[0], 28 *
28)).astype('float32') / 255
test_images = test_images.reshape((test_images.shape[0], 28 *
28)).astype('float32') / 255
```

TENSORFLOW

```
# Build the model
model = Sequential([
    Dense(128, activation='relu', input_shape=(28 * 28,)),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# Train the model
model.fit(train_images, train_labels, epochs=5)

# Evaluate the model
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc}")
```

TENSERFLOW

```
Epoch 1/5
1875/1875 ————— 7s 3ms/step - accuracy: 0.8771 - loss: 0.4387
Epoch 2/5
1875/1875 ————— 11s 3ms/step - accuracy: 0.9632 - loss: 0.1214
Epoch 3/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9758 - loss: 0.0792
Epoch 4/5
1875/1875 ————— 7s 3ms/step - accuracy: 0.9829 - loss: 0.0563
Epoch 5/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9869 - loss: 0.0442
313/313 ————— 1s 2ms/step - accuracy: 0.9727 - loss: 0.0869
Test accuracy: 0.9769999980926514
```

EXAMPLE 1

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Generate synthetic data
x_train = np.random.rand(1000, 10)
y_train = np.random.randint(2, size=(1000,))
x_test = np.random.rand(200, 10)
y_test = np.random.randint(2, size=(200,))

# Define a simple binary classification model
model = keras.Sequential([
    keras.layers.Dense(32, activation='relu', input_shape=(10,)),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])
```

EXAMPLE 1

```
# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=10, batch_size=32, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc:.4f}")
|
```

EXAMPLE2

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Create training data (simple linear function  $y = 2x + 1$ )
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=np.float32)
y = np.array([3, 5, 7, 9, 11, 13, 15, 17, 19, 21], dtype=np.float32)

# Define a simple neural network model
model = keras.Sequential([
    keras.layers.Dense(1, input_shape=(1,))
])
```

EXAMPLE2

```
# Compile the model
model.compile(optimizer='sgd', loss='mean_squared_error')

# Train the model
model.fit(X, y, epochs=500, verbose=0)

# Make predictions
predictions = model.predict([11, 12, 13, 14, 15])
print(f'Predicted values: {predictions.flatten()}')
```